

HTML and Javascript: Lab worksheet H1

Overview of learning objectives for HTML and Javascript component

There are initially 3 lab sessions in weeks 1-3 (one per week) associated with the HTML and Javascript component of ENG0018. The learning objectives (LO) and content of the learning materials in these sessions are closely linked to the Conference Project *paper* component. There will be some further lab sessions on HTML and JavaScript later in the semester, in weeks 9 and 10, focusing on the formatting for the online *presentation* component of the conference project.

The *aim* of the initial 3 lab sessions in HTML and JavaScript is to equip you with the skills necessary to set up your conference paper on your own public webpage, deployed through the GitHub platform. The general learning objectives for each of the first three lab sessions are summarised in the table below.

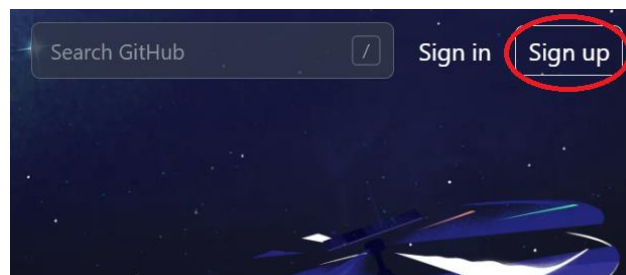
Lab/week	Learning Objectives
H1(1)	1.1 Create a GitHub account and apply settings for initial webpage deployment 1.2 Gain familiarity with html editing on GitHub
H2(2)	2.1 Design webpage layout and organisation of information 2.2 Format text for the web 2.3 Format tables 2.4 Insert hyperlinks to external URLs such as embedded references
H3(3)	3.1 Embed JavaScript in html to create more sophisticated webpage functionality. 3.2 Use JavaScript to create and customise a chart using the Google chart libraries. 3.3 Implement a time-stamp feature for submission validation 3.4 Implement section or document word count

These learning objectives will be assessed later in the semester as a part of the formatting component of your web-based conference paper submission.

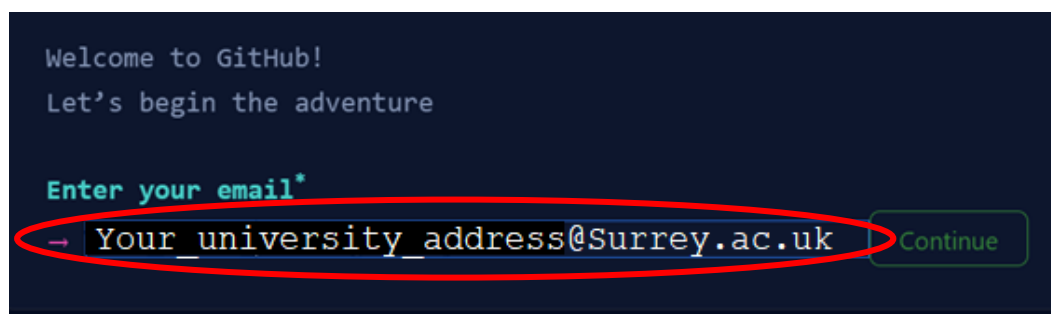
The remainder of this worksheet focuses on H1 learning objectives 1.1 - 1.2

LO 1.1 Create a GitHub account and apply settings for initial webpage deployment

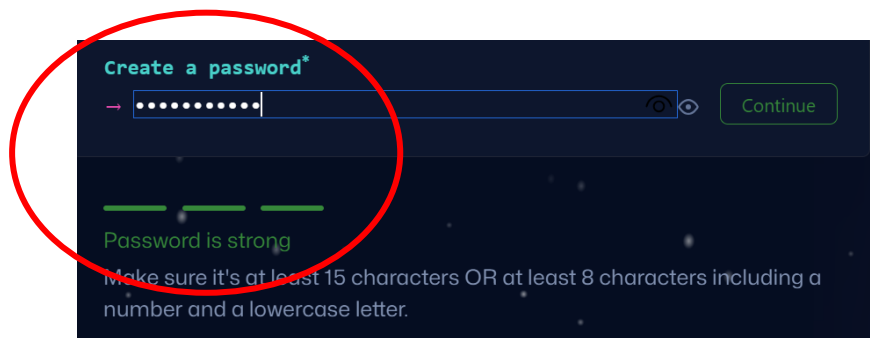
1. We will use GitHub to host our webpages. GitHub offers a great deal of functionality for developers working on large, collaborative software projects. The user interface may appear complicated at first, but we will only be using a tiny fraction of the functionality to create and host our webpages. Before we can do anything though, we need to set up individual GitHub accounts. You must follow the instructions in the following account set-up steps carefully, otherwise we may not be able to access or assess your work.
2. Using the following link, open the [GitHub web page](#) and in the top right, click on the “Sign up” button.



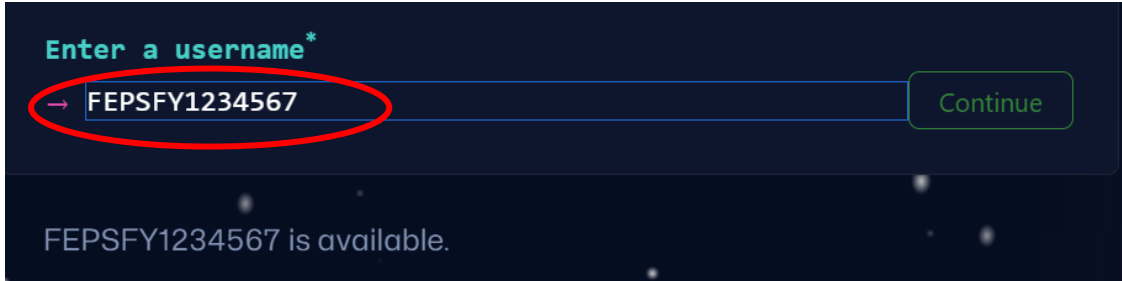
3. Enter your University of Surrey email address. **Do not use your personal email address.** If you already have a GitHub account with a personal email address, you will need to create a new one using your University of Surrey email address.



4. Create a **strong password** for your account. The password should satisfy the following criteria: 15 characters or alternatively 8 characters including a number and a lowercase letter. Symbolic characters may also be used. There will be an indicator showing when your password is strong.



5. Enter a username. **Your username must take the format FEPSFY#####** where the ##### symbols represent your 7-digit student URN (ID number), as shown in the example below. Replace 1234567 in the example with your own URN.

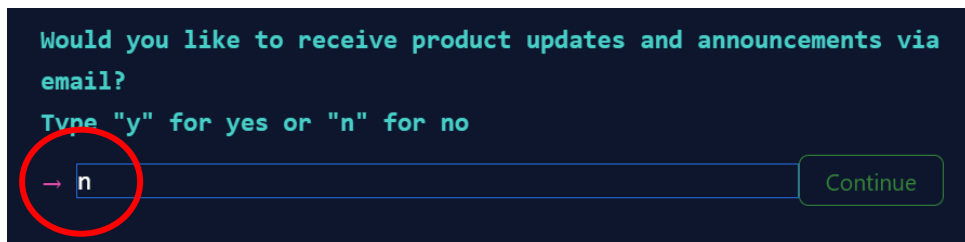


Enter a username*

→ FEPSFY1234567 Continue

FEPSFY1234567 is available.

6. Enter "n" for the product updates, we will not need this.

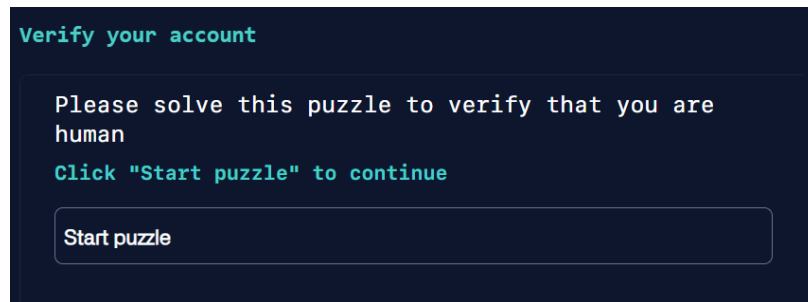


Would you like to receive product updates and announcements via email?

Type "y" for yes or "n" for no

→ n Continue

7. You will be prompted to solve two puzzles as part of the account verification process. After successfully completing these puzzles, you'll be taken to some personalisation options.



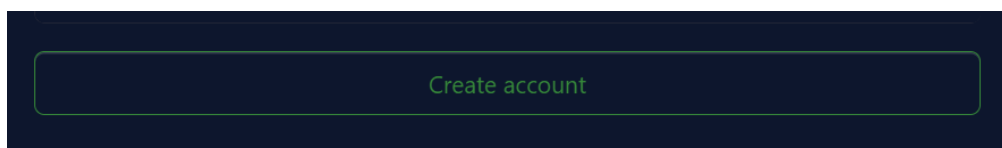
Verify your account

Please solve this puzzle to verify that you are human

Click "Start puzzle" to continue

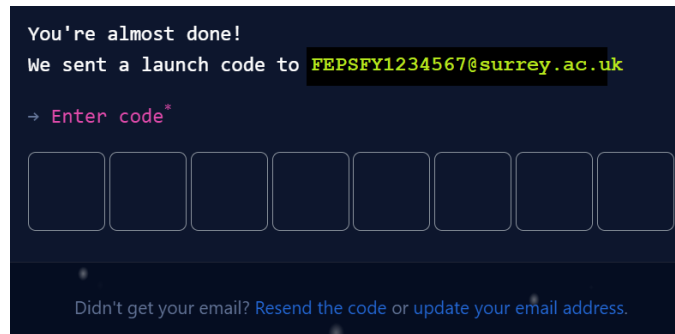
Start puzzle

8. Click on the create account button.

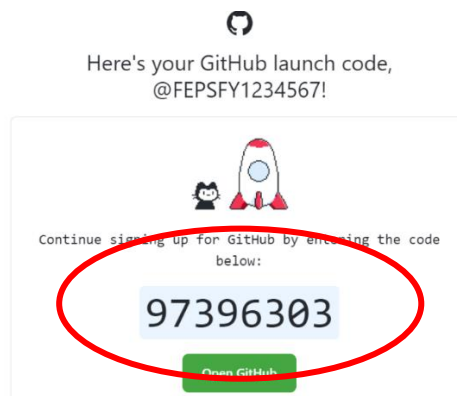


Create account

9. Clicking on create account will trigger a verification email to be sent to your University of Surrey email account, it may appear in the “Other” or Junk Email folder – you should check these if it doesn’t appear directly in the “Focused” folder of your inbox. On GitHub you will see the prompt to enter the verification code (which will be in the email.)



10. Get your launch code from the email – there is no need to select “Open GitHub” as you should already have GitHub open in another window. Your launch code will be different to the example shown.



11. The first personalisation option asks you about team members. Select the “Just me” option and “student” – then click on the “continue” button.

How many team members will be working with you?

This will help us guide you to the tools that are best suited for your projects.

Just me 2 - 5 5 - 10

10 - 20 20 - 50 50+

Are you a student or teacher?

Student Teacher

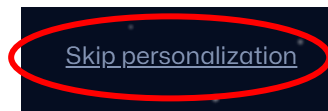
Continue

12. When the next personalisation page opens don't select anything (do not select "continue") scroll down to the bottom of the page and click on the "Skip personalisation."

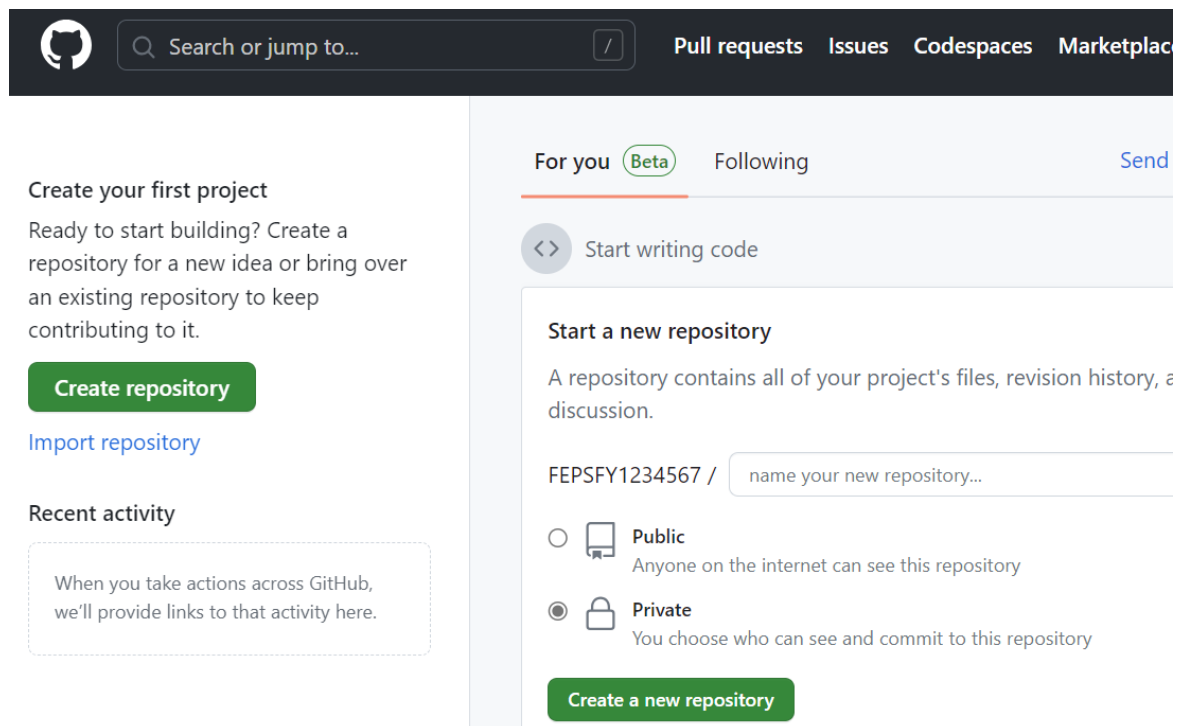
What specific features are you interested in using?

Select all that apply so we can point you to the right GitHub plan.

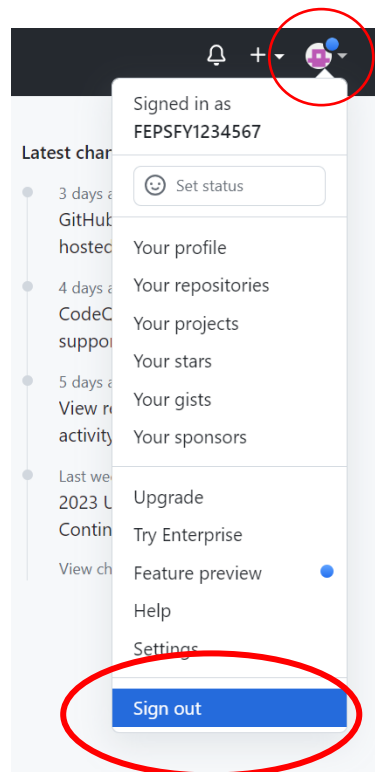
- ☐  Collaborative coding
- Codespaces, Pull requests, Notifications, Code review, Code review assignments, Code owners, Draft pull requests, Protected branches, and more.



13. You'll now arrive at your GitHub landing page as shown below. Do not select any options on this page for now, we will first check that our basic account is set up correctly.



14. Click on the icon in the top right of the page and a drop-down menu will appear. Select the “Sign out” option at the bottom of the menu to log out of your GitHub account.



15. Now we will sign back in again to verify our account is working OK and get ourselves set up ready to deploy our very first public webpage! Select the sign in option at the top right. Using your username and password, sign into your account.



Sign in to GitHub

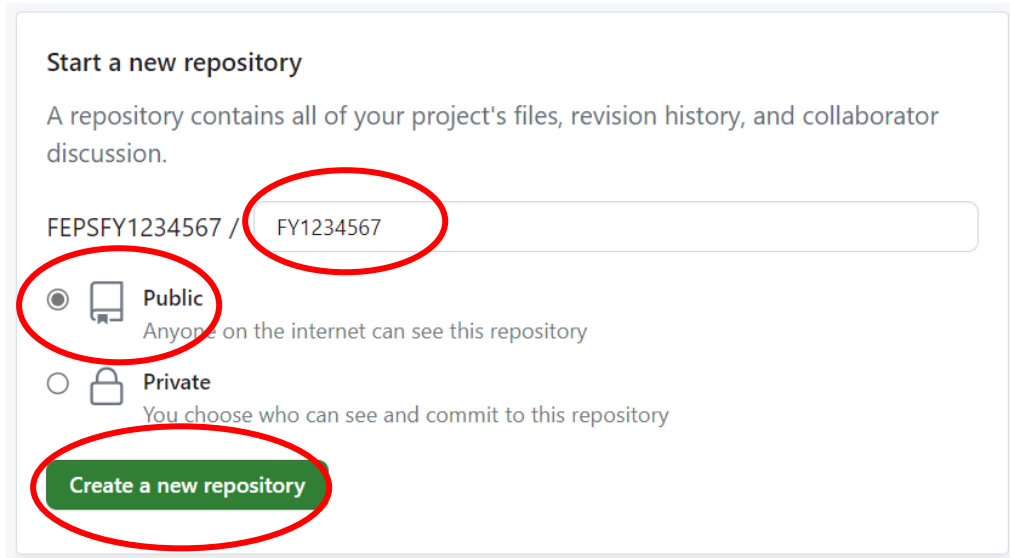
Username or email address

Password

Forgot password?

Sign in

16. Before we go about generating our very first public webpage, we need to set up a few basic parameters. The first step is to create a repository. The name of the repository **must be FY#####** where the # symbols are your 7-digit student ID number. Go ahead and name your repository, replacing 1234567 in the example below with your own URN. Then select the “public” option and click on the “Create a new repository” button.



Start a new repository

A repository contains all of your project's files, revision history, and collaborator discussion.

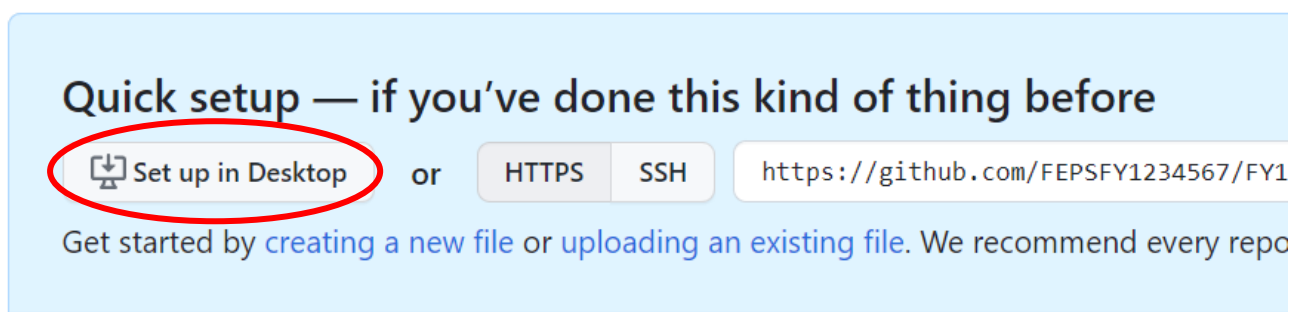
FEPSFY1234567 / FY1234567

☒  **Public**
Anyone on the internet can see this repository

☐  **Private**
You choose who can see and commit to this repository

Create a new repository

17. Although steps 17-26 are not essential, we can also now setup GitHub desktop (you can do this on your own laptop/Macbook or PC at home later.) We will primarily be using the web-based application and not making that much use of the desktop version but it's convenient to get the desktop application set up at this point in case you want to use it at some point later. In the Quick setup, click on the “Set up in desktop” button.



Quick setup — if you've done this kind of thing before

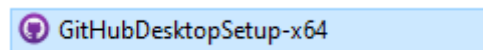
 **Set up in Desktop** or **HTTPS** **SSH** <https://github.com/FEPSFY1234567/FY1>

Get started by [creating a new file](#) or [uploading an existing file](#). We recommend every repo

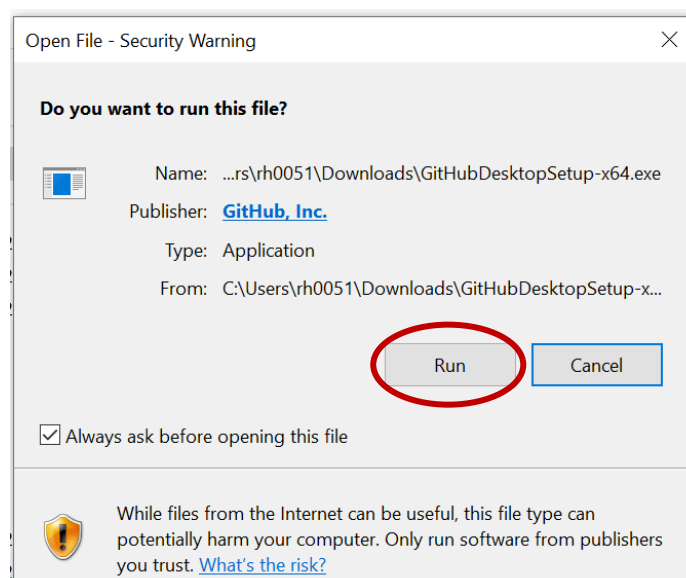
18. You should now be on the desktop app download page. Click on the “Download for Windows” button. If you are setting up on your own device (recommended) choose to download the app which is applicable to your device, e.g MacOS.



19. The installation file we just downloaded will (probably) by default be located in the download folder on the hard drive of the lab machine you are working on. Navigate to the PC download folder and check that the desktop installer is there. It will have an icon similar to the one shown below. If you cannot locate it, ask a demonstrator or the tutor for help.



20. Before doing the installation you need to copy and paste the downloaded file into your own **University of Surrey OneDrive** area. You should already have a folder set up in your OneDrive area for the computer laboratory module. Move the downloaded file to this folder, then double-click on the file to run the desktop installer.

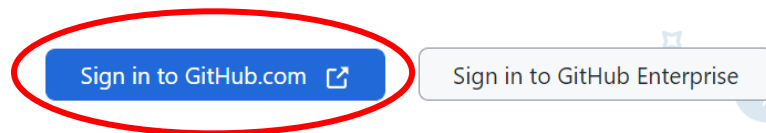


21. You should now see a new window welcoming you to the GitHub desktop. Sign in again using your account details.

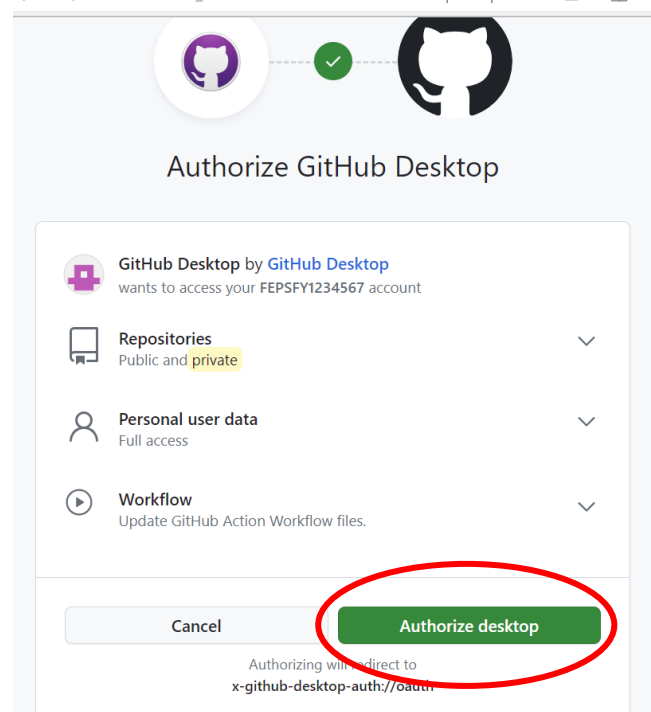
Welcome to GitHub Desktop

GitHub Desktop is a seamless way to contribute to projects on GitHub and GitHub Enterprise. Sign in below to get started with your existing projects.

New to GitHub? [Create your free account.](#)



22. We now have to authorise the desktop version of GitHub – click on the “Authorise desktop” button. If another pop-up window appears with the message “This site is trying to open GitHub desktop” – click on the “Open” button.



23. The next step is to configure “Git.” Git is the component of **GitHub** used to store the source code for a software project. You can accept the default information that appears and select “Use my GitHub account name and email address.” Then click on the “finish” button.

Configure Git

This is used to identify the commits you create. Anyone will be able to see this information if you publish commits.

☒ Use my GitHub account name and email address
☐ Configure manually

Name

FEPSFY1234567

Email

135635755+FEPSFY1234567@users.noreply.github.com

Example commit

Fix all the things

FEPSFY1234567 • 30 minutes ago

24. You should now see the “Let’s get started!” page. First clone the repository that we created online earlier. Select “Clone a repository from the internet” then click on the blue “Clone FEPSFY...” button. If there is no blue button, just click on the repository name as circled in red below.

File Edit View Repository Branch Help

Let's get started!

Add a repository to GitHub Desktop to start collaborating

Create a tutorial repository...

Filter your repositories

your repositories

FEPSFY1234567/FY1234567

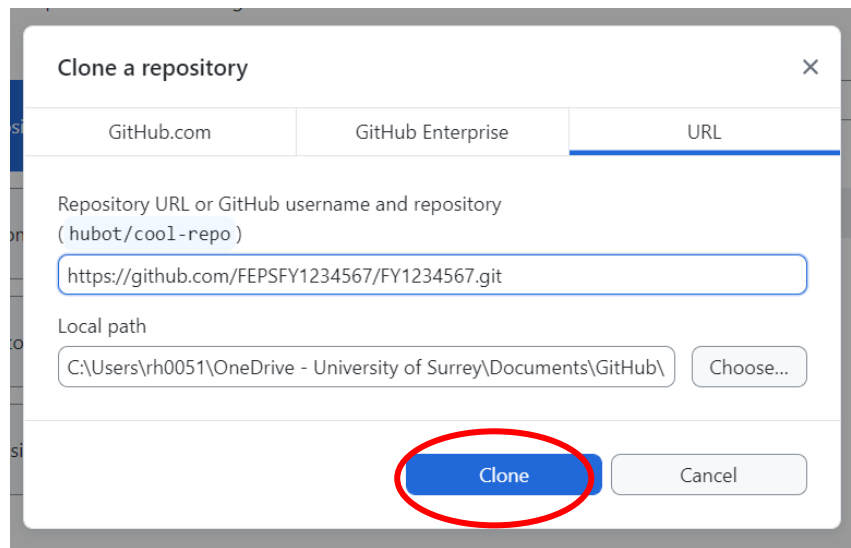
Clone a repository from the Internet...

Create a New Repository on your hard drive...

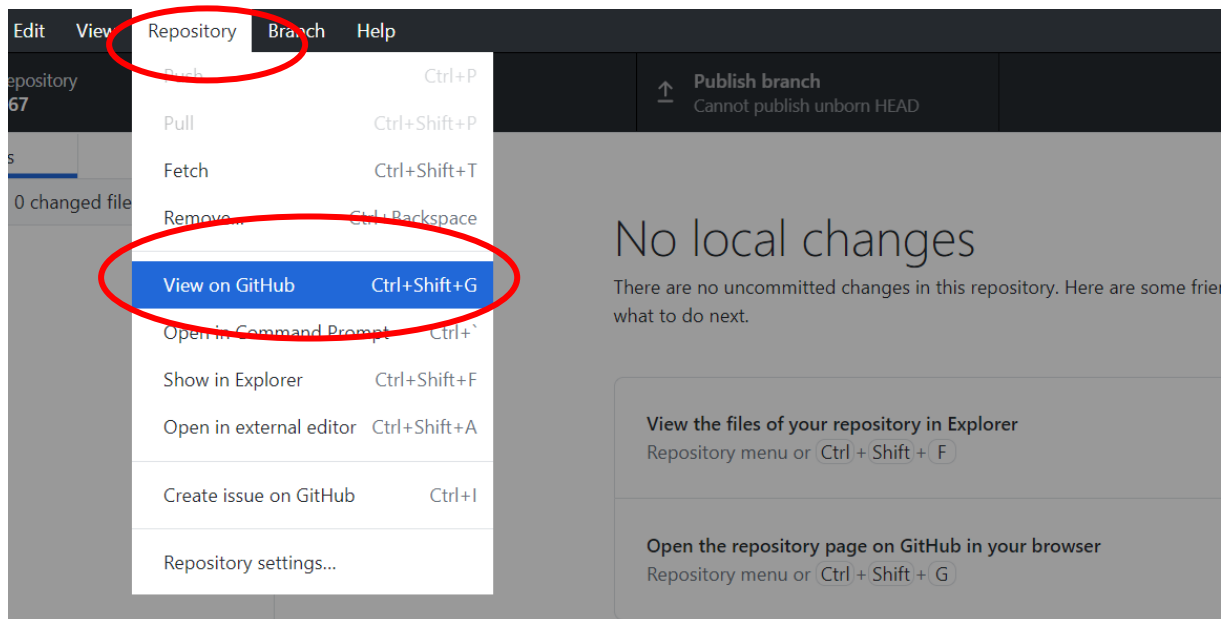
Add an Existing Repository from your hard drive...

Clone FEPSFY1234567/FY1234567

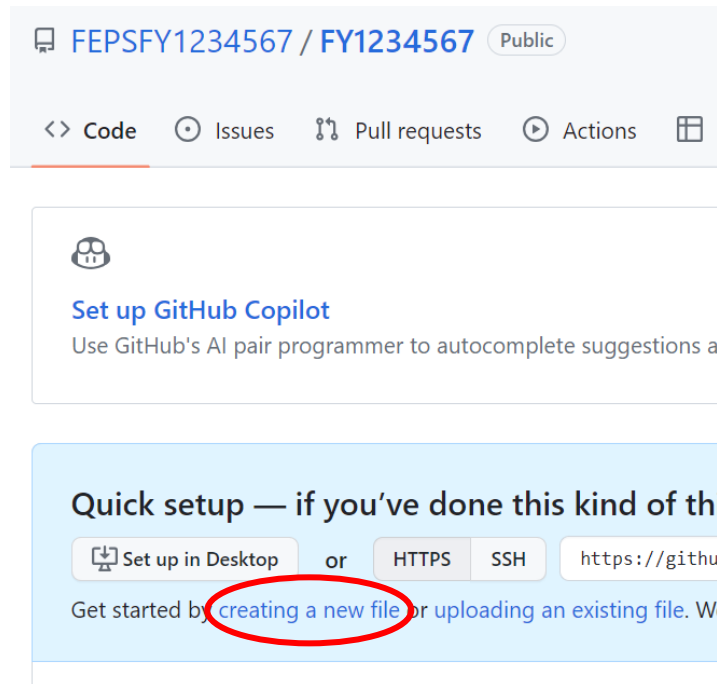
25. Another pop-up window will appear (it may be slightly different too the one shown below.) Click on the “Clone” button, accepting the default information that appears in the window.



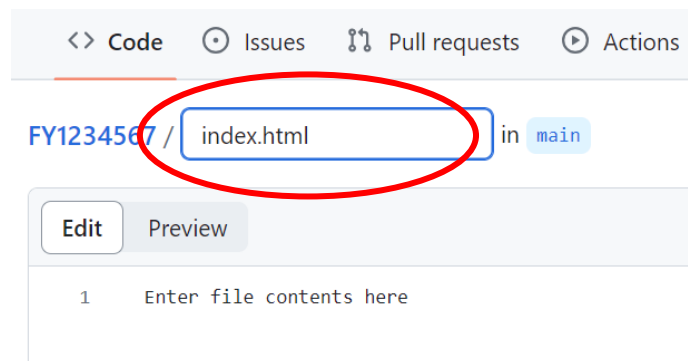
26. Now we will view our repository on GitHub. Click on the “Repository” tab and select the “View on GitHub” option from the drop-down menu.



27. Finally we can create a file and start to write some html for our web page! Select the “Creating a new file” option



28. Your file **must** be named as “index.html” – type this into the box.



29. Writing your first html code - html stands for *Hyper-Text Markup Language*. It is a worldwide standard language for website construction. Html uses what are called tags, contained in angled brackets `< >`. The tags define the formatting of the page. In simple terms, the **content** of a webpage is displayed in a **browser** and the html tags tell the browser **how to display the content**. The tags themselves and any **comments** are not displayed on the webpage. You will learn more about html in the next lab session H2 but for now here is some basic information to get us started on creating our first webpage.

- For our purposes, html code can be written in a text file or in the editor on GitHub. An html file (or document) will usually have `<!DOCTYPE html>` as the first line of the code. The exclamation symbol “!” inside the bracket `<! -->` indicates we are writing a comment (it will not be displayed on the webpage.) Comment tags are self-contained, since the content is within the tag. Unlike almost all other tags (used for formatting), comments do not have an end tag.
- The **start** and **end** tags of an html document are `<html>` and `</html>` respectively. All of the html code and the content for a webpage will usually be contained between these two tags.
- Html pages typically have a **head** and a **body**. The body contains the main content and the head contains additional formatting/styling code as well as some content. The head and body section of the document will have start and end tags of `<head> </head>` and `<body> </body>` respectively.
- There are many different html tags for applying different formats and styling of the page content. For example `<h2> 1234567 </h2>` tells the browser to display the content – the numbers `1234567` as a “heading” of “size” 2. The content as displayed on the webpage in the browser will appear much larger, compared to the size at which it appears in the html code document.
- The start and end tags, together with the content (e.g in the above `<h2> 1234567 </h2>`) are usually called an **element**. An html document will contain many such elements which collectively create the styling of the webpage.

30. Now for the exciting part – generating your first public webpage! It is important to be aware of the following information and guidance:

Your webpage is to be used purely for the display and assessment of your conference paper and presentation. Bear in mind that anyone worldwide will be able to access your webpage.

- Do **NOT** put any personal information such as your name etc on your webpage.
- Do **NOT** put any content onto your webpage other than that which will be used to assess your conference paper and presentation.
- If you do not adhere to this guidance you could find yourself being subject to disciplinary action and your webpage might not be assessed.

In the GitHub file editor, enter the html code shown below into your html document “index.html.” You should replace the content in the example (URN 1234567) with your own content, such as your URN or the commonly used “Hello world!”

- Use the return key on the keyboard to start a new line of code
- The code should be *indented* from the left margin and relative to the html tags as shown
- You can use the tab key and backspace key to adjust the level of indentation (if it doesn’t indent automatically.)

FY1234567 / index.html in main

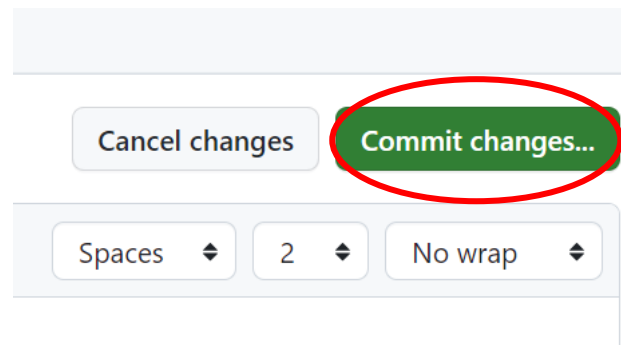
Edit
Preview

```

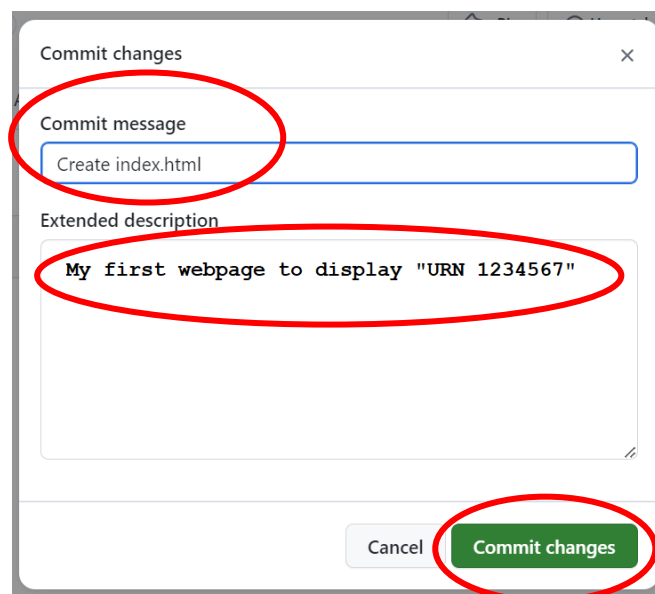
1      <!DOCTYPE html>
2
3      <html>
4
5          <head>
6              <h2>URN 1234567 </h2>
7          </head>
8          |
9      </html>
10

```

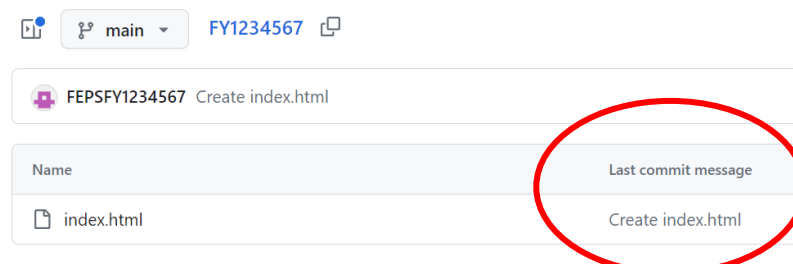
31. At this stage, we can't see our webpage – there are some steps to go through before we can see the output online. The first step is to commit the changes to the html document.



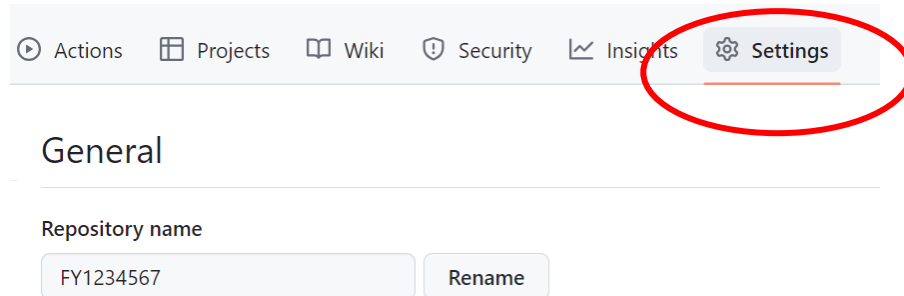
32. GitHub lets you enter notes on what changes you make to your code. It's a good idea to make a brief note so you can keep track of the changes that you made. This becomes vitally important when working collaboratively on larger development projects so it's a good idea to get into the habit of making brief notes now. You don't have to write much, just a phrase or two, as in the example below. You can also enter a "Commit message" which will be displayed after the changes are committed. After entering your notes, click on the "Commit changes" button.



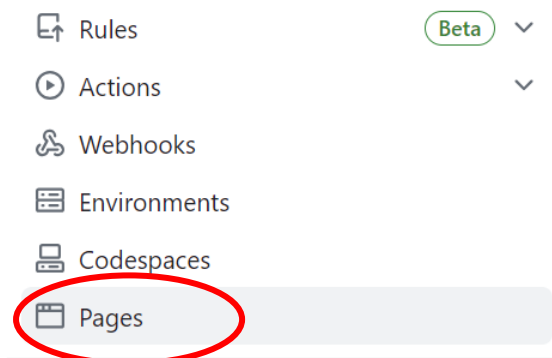
33. GitHub will confirm that the webpage has been created, displaying the commit message that you entered previously;



34. We now have to navigate our way through some additional steps in order to view the online output to the webpage. First click on the “settings”



35. Next click on “Pages” in the menu on the left hand side of the page.



36. We now have to specify the Build and Deployment settings. Change the Branch setting from “None” to “Main.”

GitHub Pages

GitHub Pages is designed to host your personal, organization, or company website.

Build and deployment

Source

Deploy from a branch

Branch

GitHub Pages is currently disabled. Select a source below

None

Save

Select branch

Select branch

main

✓ None

et access
ith read
ocument

37. Now save the settings.

GitHub Pages

GitHub Pages is designed to host your personal, organization, or

Build and deployment

Source

Deploy from a branch ▾

Branch

GitHub Pages is currently disabled. Select a source below to ena

main ▾

/ (root) ▾

Save

38. Click on “Settings” then “Pages” again (as in step 35) to refresh the page. You should now see the following message indicating that your site is live. You can access your webpage by clicking on the “Visit site” button or clicking on the URL (you can also copy and paste the URL into your browser.)

GitHub Pages

GitHub Pages is designed to host your personal, organization, or project pages from a GitHub repository.

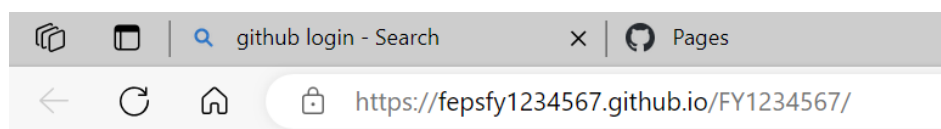
Your site is live at <https://fepsfy1234567.github.io/FY1234567/>

Last deployed by FEPSFY1234567 1 minute ago

Visit site

...

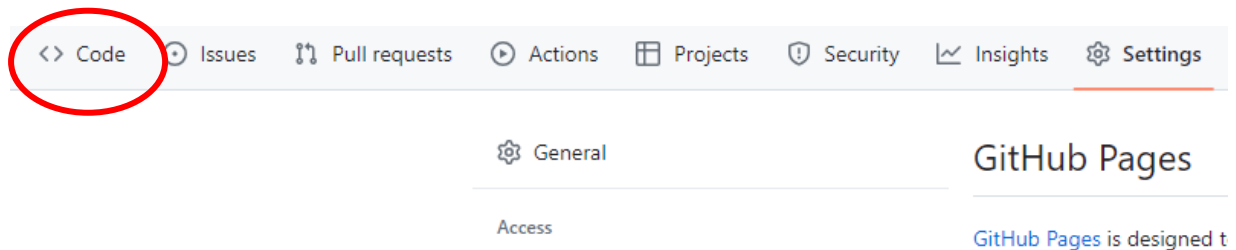
39. It sometimes takes GitHub a couple of minutes to update the webpages following changes made to the html document. It is very important to be mindful of this when you are editing and making changes to your html document. You should now be able to see your webpage displaying the content you entered between the `<h2>` `</h2>` tags. If not, let the demonstrator or tutor know.



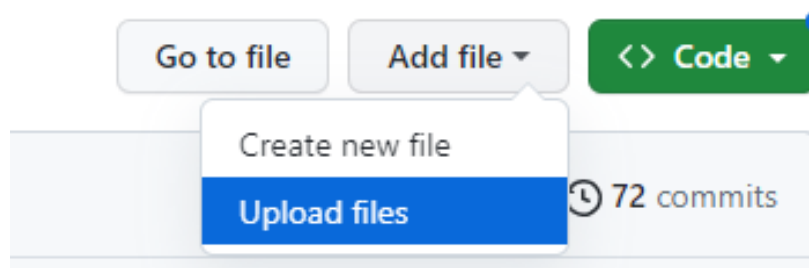
URN 1234567

LO 1.2 Gain familiarity with html editing on GitHub

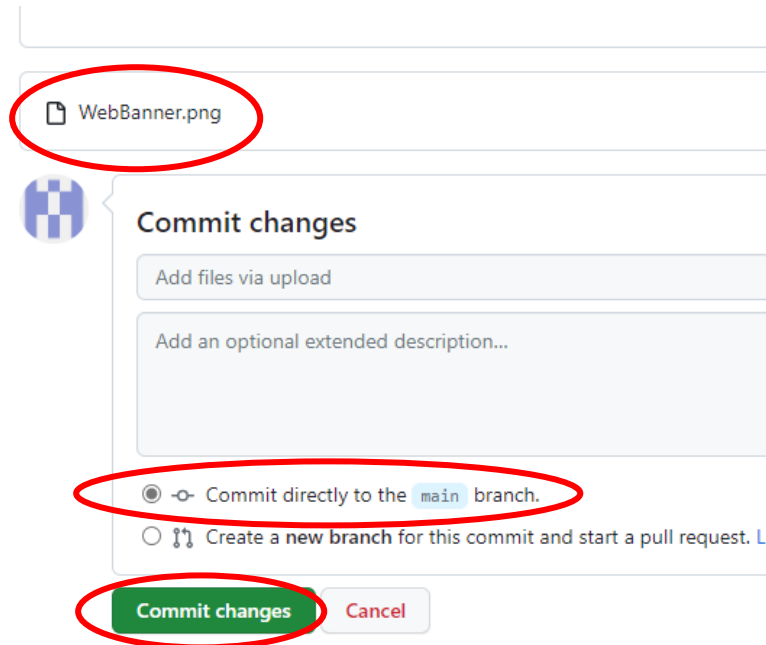
1. We have so far deployed a very basic public webpage. If we continue to add content in a similar way we will end up with a webpage looking like it was created in the early 1990's - definitely not cool, unless you're really into retro styled webpages! Many web pages have a banner at the top so we will now look at an easy way of adding the University of Surrey logo image to the top of our page. There are a few steps we need to go through; (1) acquire the image, (2) add the image to our GitHub repository, (3) edit our html file, adding code to display the image (4) check that the image is displayed. Follow the steps below to add the banner to your page.
2. Download the University of Surrey logo from Surreylearn using this link [Week 1: WebBanner2025 - COMPUTER LABORATORY \(ENG0018\) - SEMR1 2025/6 \(surrey.ac.uk\)](#). You are probably still in the "Settings/Pages" area in GitHub. Select the "<> Code" tab in the top left of the GitHub menu bar to navigate to the code page.



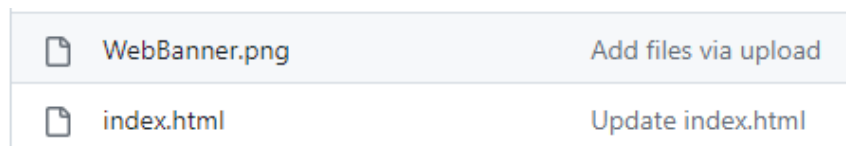
3. On the code page you can see a list of files in your repository. There will just be one file in the list at the moment, "index.html" which you created previously. We next need to get the University of Surrey banner image file "WebBanner.png" into the GitHub repository. Select the Add file/Upload files option as shown.



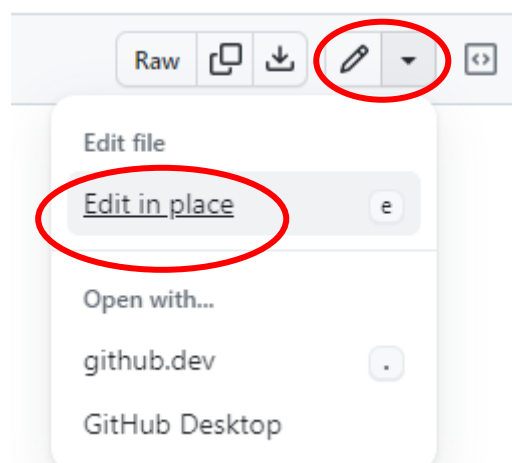
4. The upload panel will open in a new window. Drag the file “WebBanner.png” from your download folder to the upload area. The file name “WebBanner.png” will appear below the upload panel. Make sure the option “Commit directly to main branch” is selected then click the Commit changes button.



Click on the “< > Code” tab at the top and you should now see two files listed in your repository



5. Next we need to **edit** the html file and put in some additional code to display the image on the webpage. Click on “index.html” in your file list and the file containing the code will be opened in a new window. To edit the code in the file we need to select the “Edit in place” option from the drop-down menu, as shown below.



- In the editor window, delete the html element `<h2> URN 1234567 </h2>` (or whatever content you put between `<h2>...</h2>`) Now enter the new html element to display the image, `<img src = ...>` between the `<head>...</head>` tags, exactly as shown below. The coloured text identifies what the different components of the image element do.

```

```

src = source file

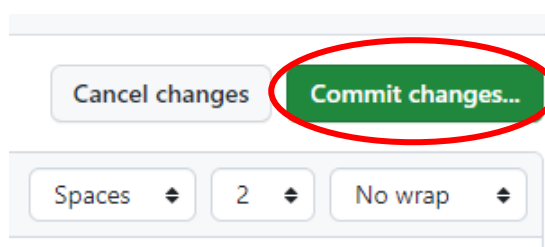
alt = alternative text if image cannot be displayed

set the image width and height so it fits on the page

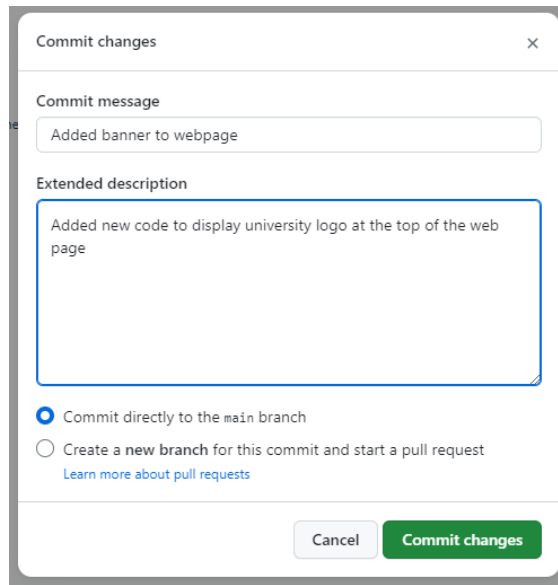
Your revised code file should now look like this

```
1 <!DOCTYPE html>
2
3 <html>
4
5 <head>
6 <head>
7 
```

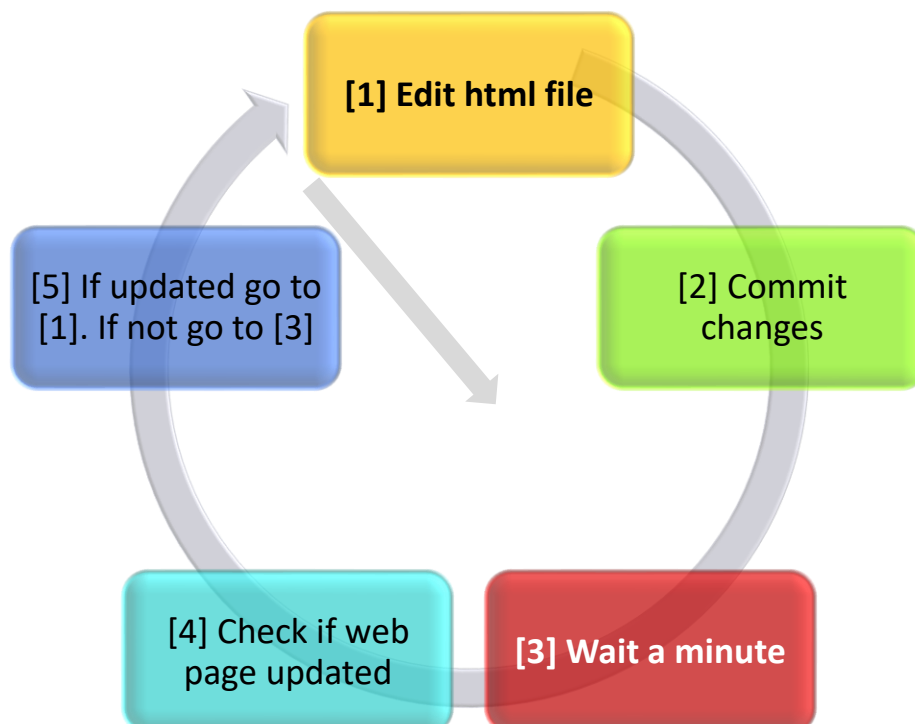
- Once you have finished with the editing, click on the “Commit changes” button.



- Whenever you edit your code and commit the changes, you will be prompted to enter some brief notes to record your changes. Fill in the boxes with your own description of what has been changed. An example is given below.



9. When you've done this, click on the commit changes button again. Remember, it can take GitHub a couple of minutes to update the changes. As a general rule, you should not edit your code further until the previous edit has been updated and displayed in the page browser, otherwise you could end up losing track of what your code file is doing in relation to what is being displayed on your web page. We will be doing many edits in the next lab sessions. A general editing procedure is summarised in the diagram below.



- 10.** To view the revised webpage, first click on “Settings” then on “Pages” in the menu on the left (see **LO 1.1** steps **34** and **35**). You’ll then be able to access your site by clicking on the URL or the Visit site button. After a while you should be able to see your page, displaying the University of Surrey logo across the top.



- 11.** That’s all for this lab session. If you have any time left at the end of the session there are several things you could be getting on with, for example;
- start thinking about the title and content for your conference paper
 - continue working on your Excel lab worksheet
 - have a look at the [W3 schools html tutorial](#) site (highly recommended!)

If you did not complete html worksheet H1 during this lab session it is expected that you will complete it in your own time before lab H2, next week.